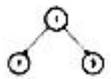
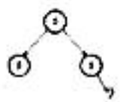
1. TREE FOUNDATIONS

TREE vs BINARY TREE vs BST

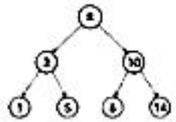
Tree
A hierarchical data structure. Each node can have 0 or more children.



Binary Tree
Each node has at most 2 children: Left and Right.

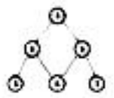


Binary Search Tree
Left < Root < Right for every node.

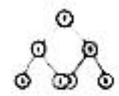


TYPES OF BINARY TREES

Full Binary Tree
Every node has 0 or 2 children.

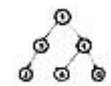


Complete Binary Tree
All levels are completely filled except possibly the last level.



Perfect Binary Tree
All internal nodes have 2 children and all leaves are at the same level.

Balanced Binary Tree
For every node, |height(left) - height(right)| <= 1



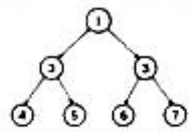
KEY TERMINOLOGY

- **Node** : Basic unit
- **Root** : Topmost node
- **Parent** : Immediate ancestor of a node
- **Child** : Immediate descendant of a node
- **Leaf Node** : Node with no children
- **Internal Node** : Node with at least one child
- **Edge** : Connection between parent and child
- **Subtree** : Tree formed by a node and its descendants
- **Height of Node** : Longest path from node to a leaf
- **Height of Tree** : Height of root node
- **Depth of Node** : Distance from root to the node (root depth = 0)
- **Level of Node** : Depth + 1

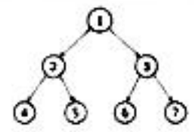
2. TRAVERSALS

DEPTH FIRST SEARCH (DFS)

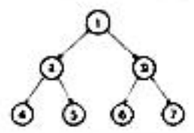
Preorder (Root → Left → Right)
AHA: Need parent first.
Use for: Build Tree, Serialize, Clone Tree
Order 1 - 2 - 4 - 5 - 3 - 6 - 7



Inorder (Left → Root → Right)
AHA: In BST, Inorder gives sorted order.
Use for: BST, Kth Smallest, Validate BST
Order 4 - 2 - 5 - 1 - 6 - 3 - 7

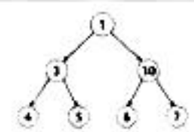


Postorder (Left → Right → Root)
AHA: Need children's answers first.
Use for: Height, Diameter, Balanced, LCA
Order 4 - 5 - 2 - 6 - 7 - 3 - 1



BREADTH FIRST SEARCH (BFS)

Level Order Traversal (Level by Level)
AHA: Need level by level.
Use for: Level Order, Right Side View, Zigzag
Order 1 - 2 - 3 - 4 - 5 - 6 - 7



DFS vs BFS

DFS (Stack / Recursion)	BFS (Queue)
Go deep first	Go level by level
Uses Stack / Recursion	Uses Queue
Good for path, subtree, properties	Good for shortest path, level based

3. TREE RECURSION TEMPLATE

THE RECURSION FLOW (FAITH → RECUR → COMBINE → RETURN)

1. **Base Case** — Check and handle the base condition.
2. **Faith** — Assume left and right subtrees will give correct answers.
3. **Recur Left** — Work on the left subtree.
4. **Recur Right** — Work on the right subtree.
5. **Combine** — Use left and right results.
6. **Return** — Return the final answer.

```
Type dfs(node):
  if node is null:
    return ...

  left  = dfs(node.left)
  right = dfs(node.right)

  // combine left and right
  ans = ...

  return ans
```

RETURN TYPE DECIDES RECURSION

Return Type	What it Means	Examples
int	Return some value (height, count, sum)	Height, Diameter, Count Nodes
boolean	Return true/false info	Same Tree, Balanced
TreeNode	Return a node	LCA
List	Return a list of values/nodes	Level Order, Paths

WHEN TO USE WHICH TRAVERSAL?

Need parent first?	→	Preorder
Need sorted order?	→	Inorder
Need children's answers?	→	Postorder
Need level by level?	→	BFS

4. DFS PATTERN PROBLEMS (POSTORDER FOCUSED)

1. MAX DEPTH
AHA: Need children's heights
Pattern: Postorder
Return: int (height)
Formula: 1 + max(left, right)
Complexity: O(n) time, O(h) space

2. SAME TREE
AHA: Both left and right subtrees must be same.
Pattern: Postorder
Return: boolean
Formula: left && right
Complexity: O(n) time, O(h) space

3. INVERT TREE
AHA: Need parent first to swap.
Pattern: Preorder
Return: TreeNode
Idea: Swap left and right, then recurse.
Complexity: O(n) time, O(h) space

4. DIAMETER OF BINARY TREE
AHA: Every node can be the center of diameter.
Pattern: Postorder
Return: int (height)
Update: diameter = max(diameter, left + right)
Complexity: O(n) time, O(h) space

5. BALANCED BINARY TREE
AHA: Every node must be balanced. Not just the root.
Pattern: Postorder
Return: int (height) + check balance
Check: abs(left - right) <= 1
Complexity: O(n) time, O(h) space

Complexity Notes
n = number of nodes
h = height of tree

Best Case (Balanced):
Time: O(n)
Space: O(log n)

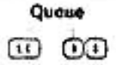
Worst Case (Skewed):
Time: O(n)
Space: O(n)

5. BFS PATTERN PROBLEMS

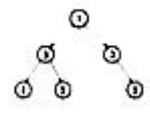
BFS TEMPLATE (LEVEL ORDER TRAVERSAL)

```
Queue<TreeNode> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
   int size = q.size();  // nodes in current level
   List<Integer> level = new ArrayList<>();

   for (int i = 0; i < size; i++) {
      TreeNode node = q.poll();
      level.add(node.val);

      if (node.left != null) q.offer(node.left);
      if (node.right != null) q.offer(node.right);
   }
   result.add(level);
}
```

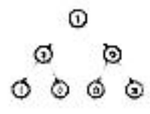
Queue



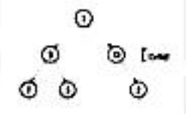
1. LEVEL ORDER TRAVERSAL (LC 102)
AHA: Need level by level.
Store nodes level by level.
Complexity: O(n) time, O(w) space (w = max width)



2. RIGHT SIDE VIEW (LC 199)
AHA: First node of each level is visible from right side.
Just take last node of each level.
Complexity: O(n) time, O(w) space



3. ZIGZAG LEVEL ORDER (LC 103)
AHA: Same as level order but alternate direction for each level.
Use flag to reverse every level.
Complexity: O(n) time, O(w) space



Note: BFS is iterative (uses Queue) and is great for level-wise problems.

6. BST BIBLE

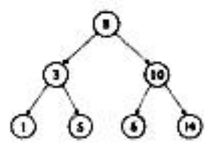
BST RULE

For every node:

Left Subtree < Root < Right Subtree

(Inorder Traversal = Sorted Order)

INODRER OF BST EXAMPLE



1 3 4 6 8 10 14

OPERATIONS IN BST

1. SEARCH
- Go left if key < node
- Go right if key > node
- Found if key == node

2. INSERT
- Go left/right till null
- Insert new node

3. DELETE (Concept)
- 0 child: delete node
- 1 child: replace with child
- 2 children: replace with inorder successor (min in right subtree) or inorder predecessor

IMPORTANT BST PROBLEMS

1. KTH SMALLEST (LC 230)
AHA: Inorder is sorted.
Do inorder and count.

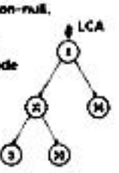
2. VALIDATE BST (LC 98)
AHA: Use range (min, max).
Not just parent comparison.
(min < node < max)

3. LCA IN BST (LC 235)
AHA: BST gives direction.
- both smaller → left
- both larger → right
- else → root is LCA

Remember: Inorder = Sorted | Use range for validation | BST gives direction

7. ADVANCED TREE PROBLEMS

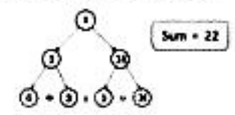
1. LCA IN BINARY TREE (LC 236)
AHA: Children report back.
If both sides return non-null, current root is LCA.
Return Type: TreeNode
Pattern: Postorder



2. BUILD TREE FROM TRAVERSALS (LC 105)
Given: Preorder + Inorder
AHA: Preorder gives root. Inorder splits left and right.
Pattern: Divide & Conquer
Complexity: O(n) time, O(n) space

```
Preorder: 1 2 4 5 3 6 7
Inorder:  4 2 5 1 6 3 7
```

3. PATH SUM (LC 112)
AHA: Subtract as you go down. If leaf and sum == 0 → true.
Pattern: DFS
Complexity: O(n) time, O(h) space



4. SERIALIZE / DESERIALIZE (LC 297) (Optional)
AHA: Convert tree to string and back.
Use Preorder + null marker.
Pattern: Preorder
Complexity: O(n) time, O(n) space

```
1,2,4,#,#,5,#,#,3,#,6,#,#
```

Mental Models
- Tree = Hierarchy
- Recursion = Explore + Combine
- Think in terms of Subproblems

Important Tip
Draw the tree. Dry run on small example.
Identify pattern. Then code.

8. TREE - ONE PAGE REVISION (AHA CHEAT SHEET)

TRAVERSAL REMINDERS

Preorder Root → Left → Right
AHA: Need parent first

Inorder Left → Root → Right
AHA: In BST, gives sorted order

Postorder Left → Right → Root
AHA: Need children's answers

BFS (Level Order)
AHA: Need level by level

KEY AHA MOMENTS

- Max Depth → 1 + max(left, right)
- Diameter → left + right (at every node)
- Balanced → abs(left - right) <= 1
- Same Tree → left && right
- Invert Tree → Swap then recurse
- Kth Smallest (BST) → Inorder + count
- Validate BST → Use range (min, max)
- LCA in BST → Use BST property
- LCA in Binary Tree → Children report back
- Build Tree → Preorder root, Inorder split

TREE vs GRAPH

TREE	GRAPH
No cycles	Can have cycles
Connected	May be disconnected
No need for visited[]	Need visited[]

COMPLEXITY QUICK VIEW

Case	Time	Space (Recursion/Queue)
Balanced Tree	O(n)	O(log n)
Skewed Tree	O(n)	O(n)
BFS (Any Tree)	O(n)	O(w) (w = max width)

FINAL MANTRA: IDENTIFY PATTERN → APPLY CORRECT TRAVERSAL → WRITE CLEAN CODE → DRY RUN → VERIFY

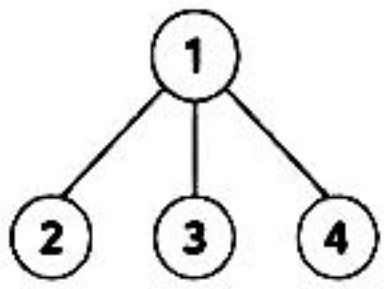
1 TREE FOUNDATIONS

The base you must be 100% clear on.

1.1 TREE vs BINARY TREE vs BINARY SEARCH TREE (BST)

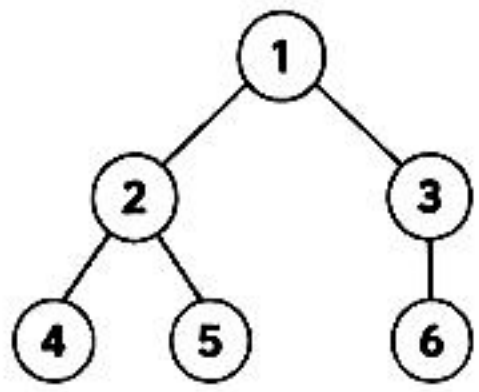
TREE

- A hierarchical structure.
- Each node can have 0 or more children.
- No ordering between child nodes.



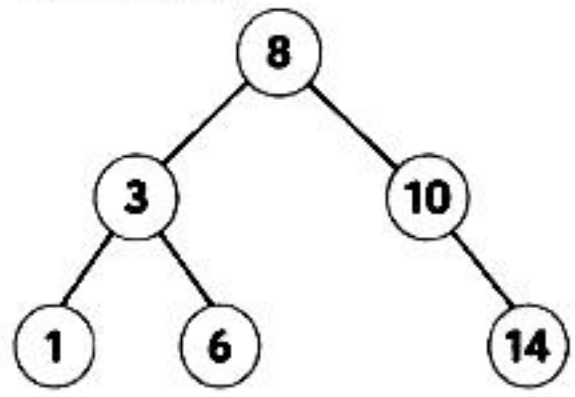
BINARY TREE

- Each node has at most 2 children: Left and Right.
- No ordering between left and right subtree.



BINARY SEARCH TREE (BST)

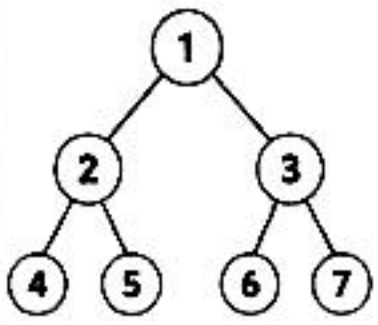
- Binary Tree + Left < Root < Right for every node.
- Inorder traversal gives sorted order.



1.2 TYPES OF BINARY TREES

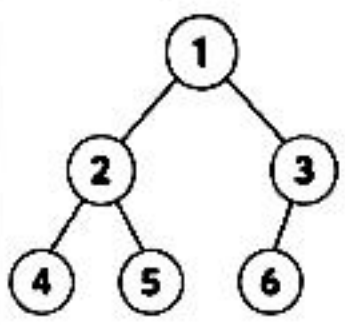
FULL BINARY TREE

Every node has 0 or 2 children.



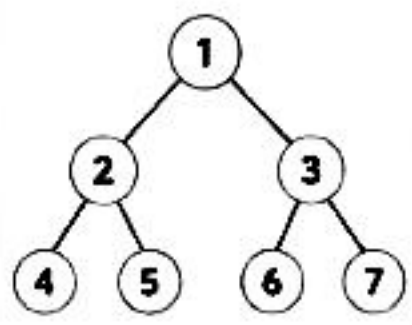
COMPLETE BINARY TREE

All levels are completely filled except possibly the last level, and all nodes are as far left as possible.



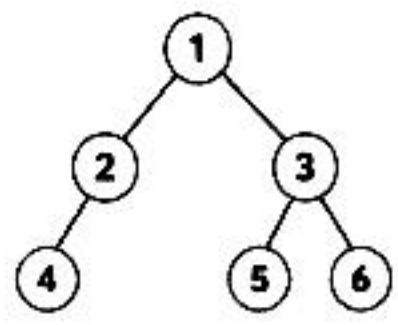
PERFECT BINARY TREE

All internal nodes have 2 children and all leaves are at the same level.



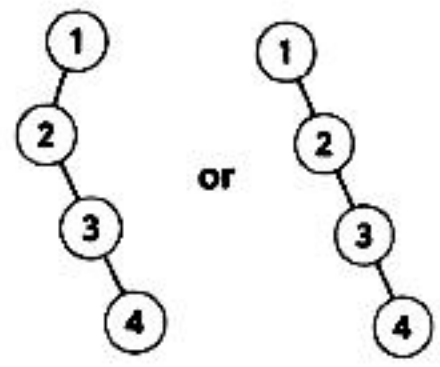
BALANCED BINARY TREE

For every node, $|height(left) - height(right)| \leq 1$



SKEWED BINARY TREE

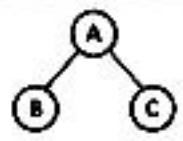
Every node has only one child. (Left Skewed or Right Skewed)

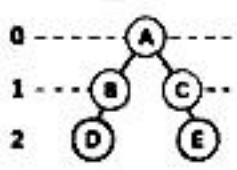


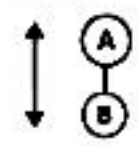
1.3 KEY TERMINOLOGY

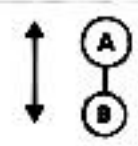
Node	:	Basic unit of a tree.
Root	:	Topmost node of the tree.
Edge	:	Connection between two nodes.
Parent	:	Immediate ancestor of a node.
Child	:	Immediate descendant of a node.
Leaf Node	:	Node with no children.
Internal Node	:	Node with at least one child.

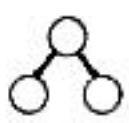
Siblings	:	Children of the same parent.
Depth of Node	:	Number of edges from root to the node. (Root depth = 0)
Level of Node	:	Depth + 1 (Root level = 1)
Height of Node	:	Number of edges on the longest path from the node to a leaf.
Height of Tree	:	Height of the root node. (Longest path from root to a leaf)
Path	:	Sequence of nodes connected by edges.
Subtree	:	A tree formed by a node and its descendants.









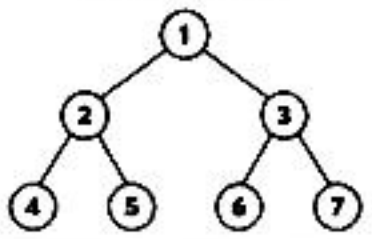


Remember: Tree is a hierarchical structure. Understanding these basics makes every tree problem easier!

2 TRAVERSALS

Ways to visit every node of a tree

QUICK VIEW



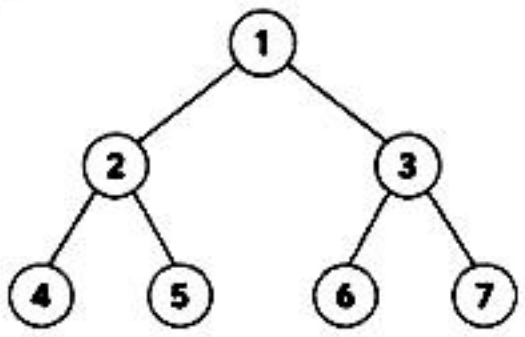
1 = Root

2,3 = Level 1

4,5,6,7 = Level 2

2.1 DEPTH FIRST SEARCH (DFS) TRAVERSALS

1 PREORDER (Root → Left → Right)



ORDER OF VISIT

1 - 2 - 4 - 5 - 3 - 6 - 7

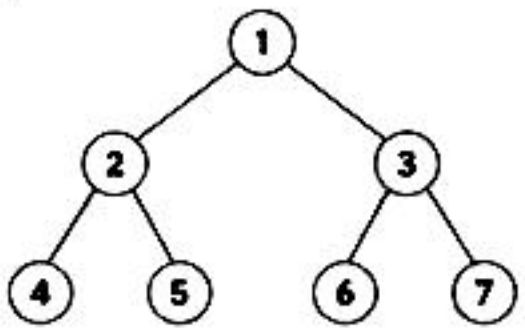
AHA MOMENT

Need parent first.
Visit root before its children.

WHEN TO USE?

- Create / Copy / Clone Tree
- Serialize Tree
- Prefix expressions
- Build Tree (with Inorder)

2 INORDER (Left → Root → Right)



ORDER OF VISIT

4 - 2 - 5 - 1 - 6 - 3 - 7

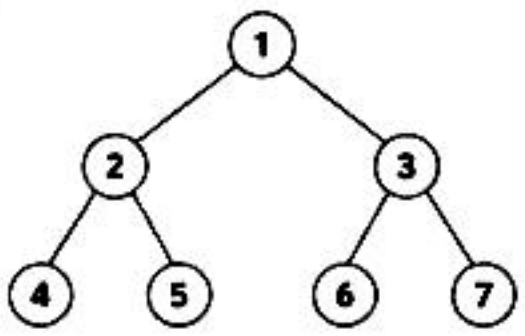
AHA MOMENT

In BST, Inorder gives sorted order.

WHEN TO USE?

- BST Problems
- Sorted order
- Kth Smallest / Largest in BST
- Validate BST (Iterative)

3 POSTORDER (Left → Right → Root)



ORDER OF VISIT

4 - 5 - 2 - 6 - 7 - 3 - 1

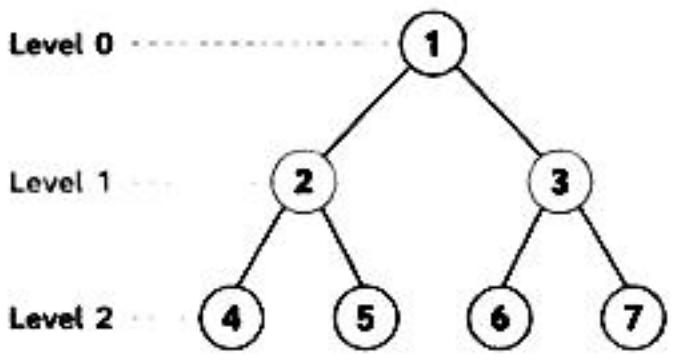
AHA MOMENT

Need children's answers first.
Process children before parent.

WHEN TO USE?

- Delete / Free Tree
- Postfix expressions
- Evaluate expressions
- Height, Diameter, Balanced, LCA, Path Sum (etc.)

2.2 BREADTH FIRST SEARCH (BFS) TRAVERSAL (LEVEL ORDER)



ORDER OF VISIT (LEVEL ORDER)

1 - 2 - 3 - 4 - 5 - 6 - 7

AHA MOMENT

Need level by level.
Use Queue.
Queue size tells current level.

TEMPLATE (BFS LEVEL ORDER)

```
Queue<Node> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
    int size = q.size();  // nodes in current level
    for (int i = 0; i < size; i++) {
        Node node = q.poll();
        // process node
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
}
```

2.3 DFS vs BFS

	DFS (Depth First Search)	BFS (Breadth First Search)
Strategy	Go deep to a leaf, then backtrack	Visit level by level from left to right
Data Structure	Stack (or Recursion)	Queue
Time Complexity	O(n)	O(n)
Space Complexity	O(h) (h = height of tree)	O(w) (w = max width of tree)
Best For	Path, Height, DP on trees, Backtracking	Shortest path in unweighted tree, Level wise problems

2.4 WHEN TO USE WHICH TRAVERSAL?

Need parent before children?	→	Use Preorder
Need sorted order (BST)?	→	Use Inorder
Need children's results first?	→	Use Postorder
Need level by level?	→	Use BFS
Modify / Build / Serialize Tree?	→	Mostly Preorder
Delete / Free / Evaluate?	→	Mostly Postorder

FINAL MANTRA

- Tree problems → Think DFS first.
- Level problems → Think BFS first.
- Understand the need, then choose the traversal.

n = number of nodes
h = height of tree
w = maximum width of tree

3 TREE RECURSION TEMPLATE

Master this template, solve any tree problem!

3.1 THE RECURSION FLOW (FAITH → RECUR → COMBINE → RETURN)

1	BASE CASE	Check and handle the base condition.	`if (node == null) return ...;`
2	FAITH	Assume left and right subtrees will give correct answers.	
3	RECUR LEFT	Recur for the left subtree.	`left = solve(node.left);`
4	RECUR RIGHT	Recur for the right subtree.	`right = solve(node.right);`
5	COMBINE	Use left and right results. Do the required work.	`ans = ...;`
6	RETURN	Return the final answer.	`return ans;`

UNIVERSAL TEMPLATE

```
Type solve(Node* node) {
   if (node == null) {
      return ...;              // Base Case
   }

   Type left  = solve(node.left);  // Recur Left
   Type right = solve(node.right); // Recur Right

   Type ans = ...;                 // Combine
   return ans;                     // Return
}
```

KEY IDEA

- Break the problem into left and right subproblems.
- Solve both (faith).
- Combine their results.
- Return the final answer up the recursion stack.

FAITH → You assume your recursive calls (left & right) are correct and return the right answer.

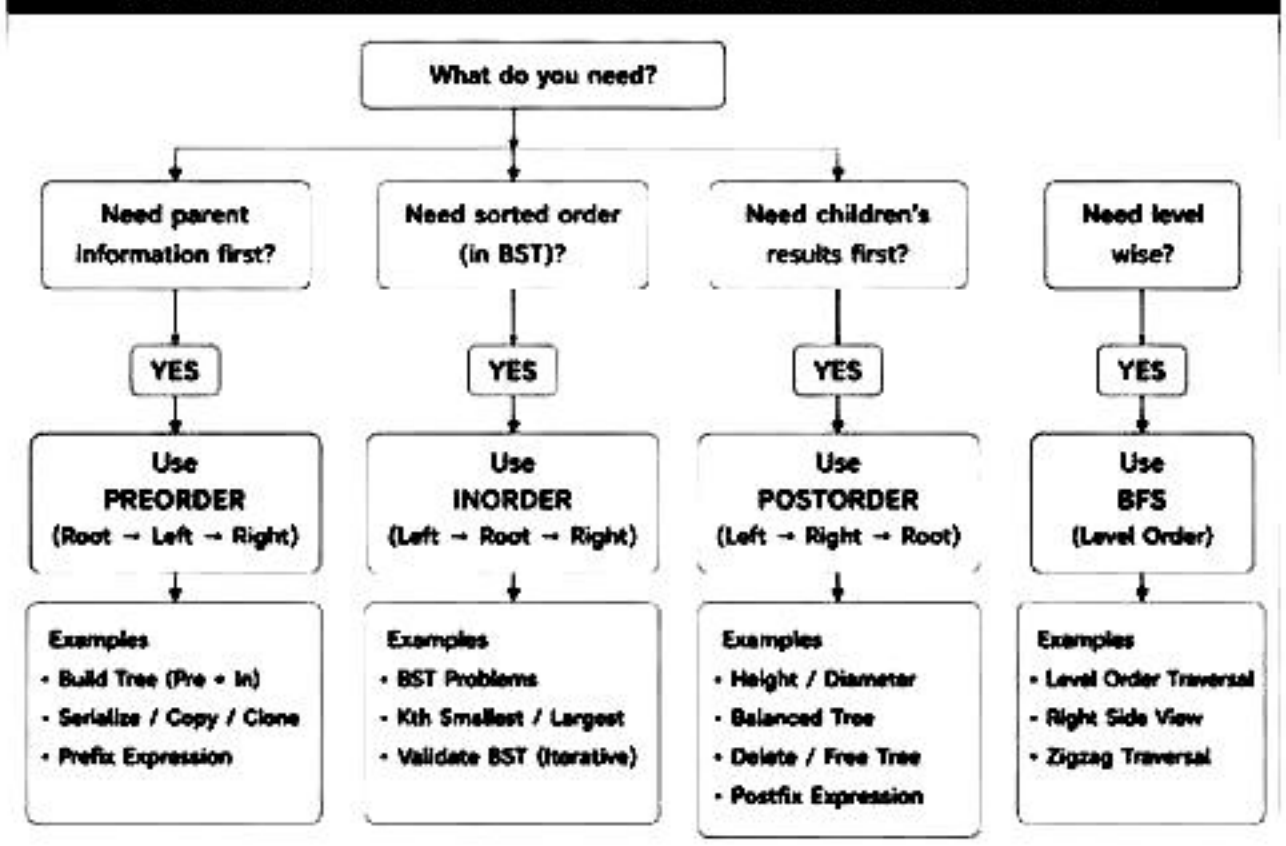
3.2 RETURN TYPE DECIDES THE RECURSION

Return Type	What it Means	Examples
int	Return some value (height, count, sum, etc.)	Max Depth, Diameter, Sum of Nodes
boolean	Return true / false	Same Tree, Balanced Tree, Valid BST
TreeNode*	Return a node	LCA, Invert Tree, Search in BST
List / Vector	Return a list of values / nodes	All Paths, Level Order, Right Side View
void	Do something (print / modify)	Print Traversals, Populate Result

3.3 WHICH TRAVERSAL TO USE?

Goal	Need	Traversal	Why?
Need parent before children?	Parent info first	Preorder (Root → Left → Right)	Visit root before going to children.
Need sorted order (in BST)?	Left first, then root, then right	Inorder (Left → Root → Right)	In BST, Inorder gives sorted order.
Need children's answers first?	Children results before parent	Postorder (Left → Right → Root)	Process children before the root.
Need level by level?	Level wise processing	BFS (Level Order)	Visit nodes level by level using Queue.

3.4 PREORDER vs INORDER vs POSTORDER (DECISION GUIDE)



REMEMBER

- Understand the need, choose the traversal.
- Wrong traversal = more code / more mistakes.
- Right traversal = simple, clean, optimal.

3.5 COMMON BASE CASES

`if (node == null) return ...;`	Used in almost all problems. Empty subtree.
`if (node.left == null && node.right == null) return ...;`	Leaf node condition.
`if (node == target) return ...;`	Found the target node.
`if (k == 0) return ...;`	Base case for depth / distance based problems.
`if (low > high) return ...;`	Used in tree construction from array / traversal.

3.6 TIPS TO MASTER RECURSION

1. Draw the tree and try dry run.
2. Identify what you need at each node.
3. Decide return type.
4. Write the template first.
5. Fill the combine logic.
6. Dry run again and verify.

GOLDEN RULE

Think small,
Recur small,
Combine smartly,
Return clearly.

FINAL MANTRA

Understand the problem → Choose the right traversal → Follow the template → Combine correctly → Return the right answer.

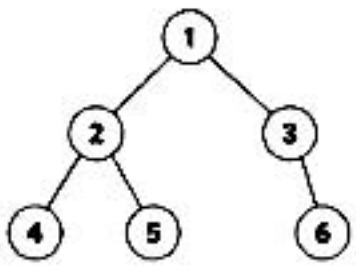
4 DFS PATTERN PROBLEMS (POSTORDER FOCUSED)

Most important tree problems follow DFS (Postorder). Master these patterns!

AHA = Key Insight | Pattern = How to think | Formula = How to compute | Complexity = Time & Space

1. MAX DEPTH OF BINARY TREE

Example



Output: 3

AHA

Need the height of left and right subtrees first, then take max. That's Postorder.

Pattern

Postorder
(Left → Right → Root)

Formula

```
1 + max(left, right)
```

Base Case:

```
if (node == null)
  return 0;
```

Code (Java)

```
int maxDepth(TreeNode root) {
  if (root == null) return 0;     // Base Case
  int left = maxDepth(root.left); // Left
  int right = maxDepth(root.right); // Right
  return 1 + Math.max(left, right); // Combine
}
```

Complexity

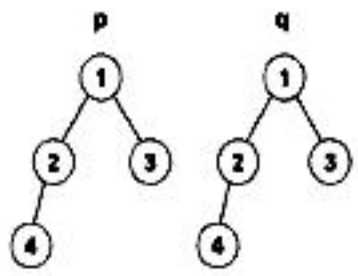
Time: O(n)

Space: O(h)

(h = height of tree)

2. SAME TREE

Example



Output: true

AHA

Both trees must match at current node, left subtree and right subtree. Return boolean.

Pattern

Postorder
(Left → Right → Root)

Formula

```
Left && Right
```

Base Case:

Both null → true

One null → false

Values not equal → false

```
boolean isSameTree(TreeNode p, TreeNode q) {
  if (p == null && q == null) return true;
  if (p == null || q == null) return false;
  if (p.val != q.val) return false;
  boolean left = isSameTree(p.left, q.left);
  boolean right = isSameTree(p.right, q.right);
  return left && right;  // Combine
}
```

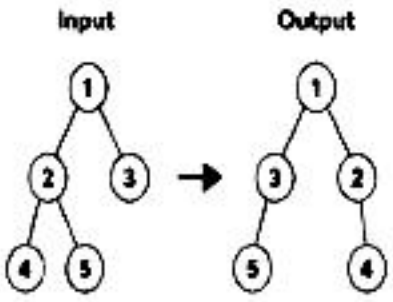
Complexity

Time: O(n)

Space: O(h)

3. INVERT BINARY TREE

Example



AHA

Need to process parent first, then swap children and recurse. That's Preorder.

Pattern

Preorder
(Root → Left → Right)

Formula

```
Swap Left and Right
```

Base Case:

```
if (node == null)
  return null;
```

Code (Java)

```
TreeNode invertTree(TreeNode root) {
  if (root == null) return null; // Base Case
  TreeNode temp = root.left;     // Swap
  root.left = root.right;
  root.right = temp;
  invertTree(root.left);         // Left
  invertTree(root.right);        // Right
  return root;
}
```

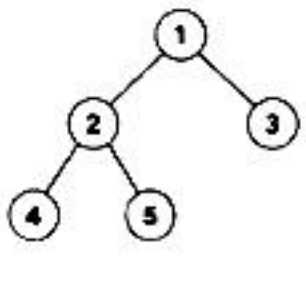
Complexity

Time: O(n)

Space: O(h)

4. DIAMETER OF BINARY TREE

Example



Output: 3
(Path: 4 → 2 → 1 → 3)

AHA

Diameter can pass through any node. Need left height and right height at every node. Update global answer.

Pattern

Postorder
(Left → Right → Root)

Formula

```
diameter =
max(diameter,
  left + right)

return height =
1 + max(left, right)
```

Code (Java)

```
int diameter = 0;  // global
int diameterOfBinaryTree(TreeNode root) {
  height(root);
  return diameter;
}
int height(TreeNode node) {
  if (node == null) return 0;
  int left = height(node.left);
  int right = height(node.right);
  diameter = Math.max(diameter, left + right); // update
  return 1 + Math.max(left, right);        // height
}
```

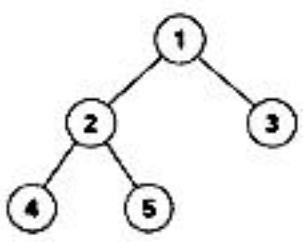
Complexity

Time: O(n)

Space: O(h)

5. BALANCED BINARY TREE

Example



Output: true

AHA

Every node must have left and right subtree heights difference <= 1. Need height of subtrees.

Pattern

Postorder
(Left → Right → Root)

Formula

```
If |left - right| > 1
return -1
(imbalance found)

Else return
1 + max(left, right)
```

Code (Java)

```
int isBalanced(TreeNode root) {
  return height(root) != -1;
}
int height(TreeNode node) {
  if (node == null) return 0;
  int left = height(node.left);
  if (left == -1) return -1;
  int right = height(node.right);
  if (right == -1) return -1;
  if (Math.abs(left - right) > 1) return -1;
  return 1 + Math.max(left, right);
}
```

Complexity

Time: O(n)

Space: O(h)

POSTORDER – WHEN TO USE?

- Need children's answers first.
- Need height / depth / size.
- Need info from left & right to compute current node.
- Delete / Free / Evaluate expressions.
- Diameter, Balanced, Max Path Sum, LCA (Binary Tree) etc.

QUICK RECAP

Height / Depth	1 + max(left, right)
Diameter	left + right (update ans)
Balanced	abs(left - right) <= 1
Same Tree	left && right
Invert Tree	Swap + Recur

COMPLEXITY LEGEND

n = number of nodes

h = height of tree

(For balanced tree: h = log n)

(For skewed tree: h = n)

FINAL MANTRA: Identify pattern → Write template → Apply formula → Handle base cases → Return correct type

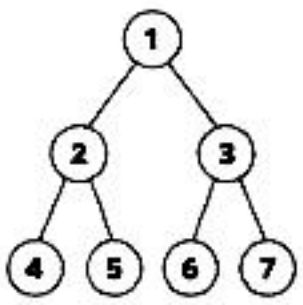
5 BFS PATTERN PROBLEMS (LEVEL ORDER FOCUSED)

BFS = Level by Level using Queue

AHA = Key Insight | Pattern = How to think | Formula / Idea | Complexity = Time & Space

1. LEVEL ORDER TRAVERSAL (LC 102)

Example



Output:
[[1], [2,3], [4,5,6,7]]

AHA

Need nodes level by level. Use Queue. Queue size tells current level.

Pattern

BFS (Level Order)

Idea / Formula

Process all nodes in queue (size = current level). Add children to queue.

BFS (Level Order) using Queue

```
List<List<Integer>> levelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>();
  if (root == null) return res;

  Queue<TreeNode> q = new LinkedList<>();
  q.offer(root);

  while (!q.isEmpty()) {
    int size = q.size();
    List<Integer> level = new ArrayList<>();

    // process current level
    for (int i = 0; i < size; i++) {
      TreeNode node = q.poll();
      level.add(node.val);

      if (node.left != null) q.offer(node.left);
      if (node.right != null) q.offer(node.right);
    }
    res.add(level);
  }
  return res;
}
```

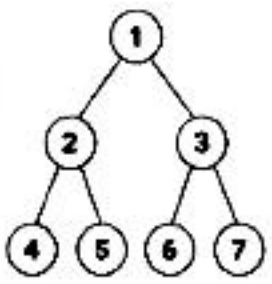
Complexity

Time: O(n)
Every node visited once.

Space: O(n)
Queue can hold max nodes of last level.

2. RIGHT SIDE VIEW (LC 199)

Example



Output: [1, 3, 7]
(Rightmost of each level)

AHA

We need the last node of each level. In each level, the last node we pop is the rightmost.

Pattern

BFS (Level Order)

Idea

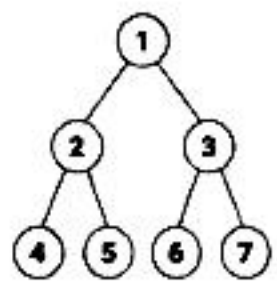
In each level, pick the last node's value.

Code (Java)

```
List<Integer> rightSideView(TreeNode root) {
  List<Integer> res = new ArrayList<>();
  if (root == null) return res;
  Queue<TreeNode> q = new LinkedList<>();
  q.offer(root);

  while (!q.isEmpty()) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
      TreeNode node = q.poll();
      if (i == size - 1) res.add(node.val);
      // last node

      if (node.left != null) q.offer(node.left);
      if (node.right != null) q.offer(node.right);
    }
  }
  return res;
}
```

3. ZIGZAG LEVEL ORDER (LC 103)

Example



Output:
[[1], [3,2], [4,5,6,7]]

AHA

Direction changes at every level. Use a flag.

Pattern

BFS + Direction Toggle

Idea

If leftToRight -> left to right
Else -> right to left

Code (Java)

```
List<List<Integer>> zigzagLevelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>();
  if (root == null) return res;
  Queue<TreeNode> q = new LinkedList<>();
  q.offer(root);
  boolean leftToRight = true;

  while (!q.isEmpty()) {
    int size = q.size();
    LinkedList<Integer> level = new LinkedList<>();

    for (int i = 0; i < size; i++) {
      TreeNode node = q.poll();
      if (leftToRight) level.addLast(node.val);
      else level.addFirst(node.val);

      if (node.left != null) q.offer(node.left);
      if (node.right != null) q.offer(node.right);
    }
    res.add(level);
    leftToRight = !leftToRight; // toggle
  }
  return res;
}
```

4. BFS TEMPLATE (REMEMBER THIS!)

```
// BFS Template
if (root == null) return;

Queue<TreeNode> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
  int size = q.size();      // nodes in current level

  for (int i = 0; i < size; i++) {
    TreeNode node = q.poll();
    // ---- Do something with node here ----

    if (node.left != null) q.offer(node.left);
    if (node.right != null) q.offer(node.right);
  } // ---- Level completed ----
}
```

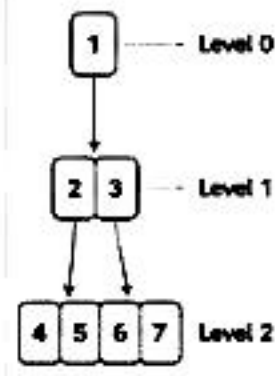
Key Points

- Use Queue
- queue.size() = current level size
- Process level by level
- Add children for next level

5. HOW QUEUE WORKS IN BFS

Step	Queue (Front → Back)	Level	Action
1	[1]	0	Start: push root
2	[] → [2, 3]	1	Pop 1, push 2, 3
3	[2, 3]	1	Process level 1
4	[] → [4, 5, 6, 7]	2	Pop 2, push 4, 5 Pop 3, push 6, 7
5	[4, 5, 6, 7]	2	Process level 2
6	[ ]	3	All nodes processed
7	[ ]	-	Queue empty, done!

Visual Flow



WHEN TO USE BFS?

- Need level by level processing
- Shortest path in unweighted graphs
- Minimum steps / transformations
- Problems involving distance / levels
- Tree view from top / side / bottom

BFS vs DFS (Quick Comparison)

Feature	BFS	DFS
Strategy	Level by Level	Go Deep, then Backtrack
Data Structure	Queue	Stack / Recursion
Best For	Level problems	Path / Depth problems
Space	O(max width)	O(max depth)

QUICK RECAP

- Use Queue for BFS.
- queue.size() gives current level.
- Process all nodes of a level together.
- Add children for next level.
- BFS = Clean + Level Order + Optimal (for shortest path problems).

FINAL MANTRA: Think Level by Level → Use Queue → Process Level → Add Children → Repeat → Get Answer

6 BST BIBLE (BINARY SEARCH TREE)

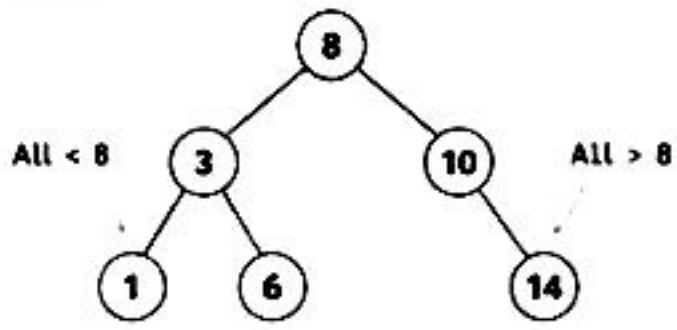
The most Important BST rules, operations and patterns

1. BST RULE

For every node N in BST

All values in Left Subtree < N < All values in Right Subtree

Example



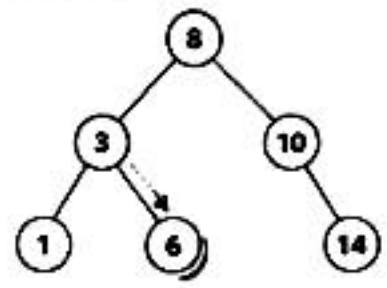
- Left subtree → smaller values
- Right subtree → larger values
- Inorder traversal gives sorted order

2. SEARCH IN BST

Idea: Use BST property to decide the direction.

```
if key == root.val      → found
else if key < root.val  → go left
else                    → go right
```

Example



Search: 6

8 > 6 → go left

3 < 6 → go right

6 == 6 → found

Complexity

Time: O(h) Space: O(1) iterative / O(h) recursive
(h = height of tree)

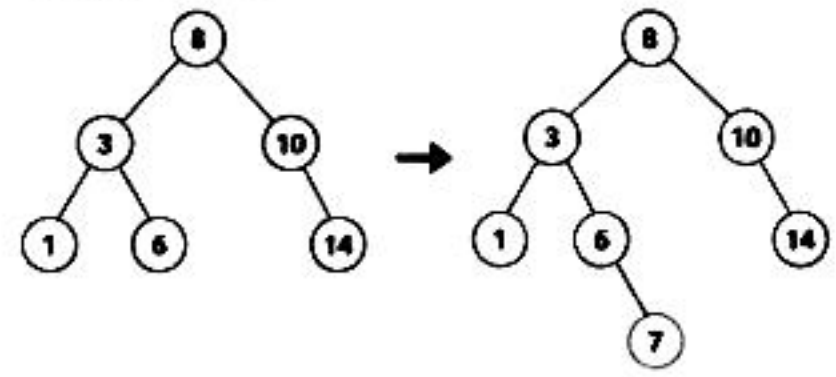
3. INSERT INTO BST

Idea: Find correct position and insert.

Steps:

1. If tree is empty → new node becomes root
2. Else compare and go left or right
3. Insert when NULL is found

Example: Insert 7



Complexity

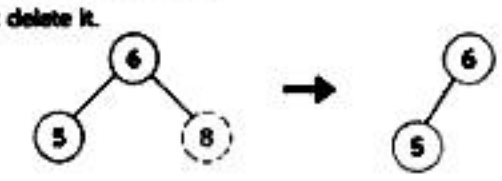
Time: O(h) Space: O(1) iterative / O(h) recursive

4. DELETE FROM BST

Cases to handle when deleting node X:

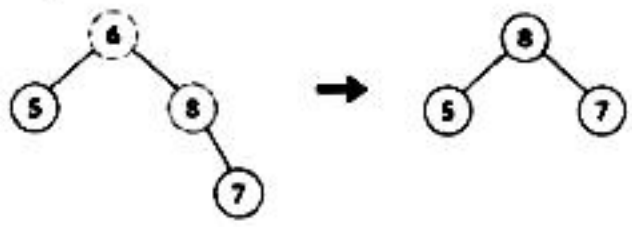
Case 1: X is a leaf node

→ Just delete it.



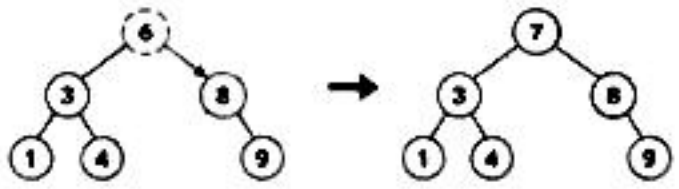
Case 2: X has one child

→ Replace X with its child.



Case 3: X has two children

→ Replace X with Inorder Successor (smallest in right subtree) or Inorder Predecessor (largest in left subtree).



Complexity

Time: O(h) Space: O(1) iterative / O(h) recursive

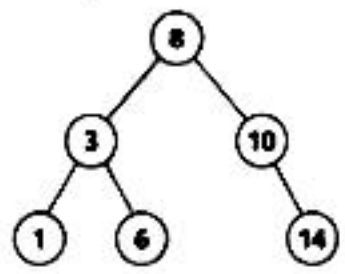
5. Kth SMALLEST ELEMENT (LC 230)

Idea: Inorder traversal of BST gives sorted order.

Steps:

1. Do Inorder (Left → Root → Right)
2. Count nodes
3. When count == k → return node

Example



Inorder: 1, 3, 6, 8, 10, 14

k = 3 → Answer = 6

Complexity

Time: O(h + k) Space: O(h)
(h = height, k = position of answer)

6. VALIDATE BST (LC 98)

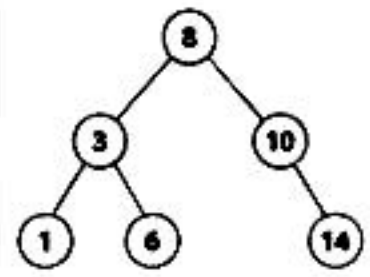
Idea: Each node must lie in a valid range.

For every node: min < node.val < max

Steps:

1. Use two bounds: min and max
2. Check: min < node.val < max
3. Recur Left → (min, node.val)
4. Recur Right → (node.val, max)

Example



For node 6:

Range (3, 8) ✓

For node 14:

Range (10, +∞) ✓

All nodes satisfy → Valid BST

Complexity

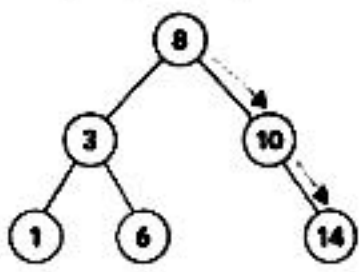
Time: O(n) Space: O(h)

7. LCA IN BST (LC 235)

Idea: Use BST property to find LCA.

If both p and q < root → go left
If both p and q > root → go right
Else → root is LCA

Example: p = 1, q = 14



1 < 8 and 14 > 8

→ 8 is LCA

Complexity

Time: O(h) Space: O(1) iterative / O(h) recursive

BST CHEAT SHEET

Operation	Idea	Time	Space
Search	Use BST property	O(h)	O(1) / O(h)
Insert	Find position & insert	O(h)	O(1) / O(h)
Delete	Handle 3 cases	O(h)	O(1) / O(h)
Kth Smallest	Inorder + count	O(h + k)	O(h)
Validate BST	Use range (min, max)	O(n)	O(h)
LCA (BST)	Use BST property	O(h)	O(1) / O(h)

BST PROPERTIES

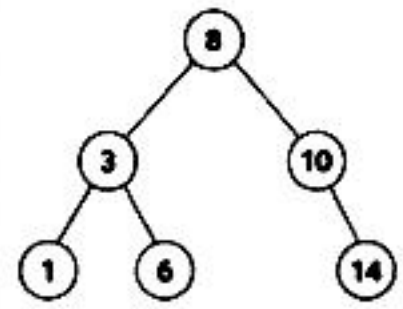
- Left < Root < Right
- Inorder = Sorted
- No duplicates (by default)
- Efficient operations: O(h)
- Balanced BST → O(log n)

REMEMBER

- Think in terms of range.
- Inorder is your best friend.
- BST gives you direction.

8. INORDER = SORTED ORDER

Proof by Example



Inorder Traversal
(Left → Root → Right)
= 1, 3, 6, 8, 10, 14
(sorted order)

Why?

All left < root < all right
So visiting Left → Root → Right
gives increasing order.

FINAL MANTRA

Understand Rule (Left < Root < Right) ➡ Use Property (Decide Direction) ➡ Apply Pattern (Search / Insert / Delete) ➡ Inorder for Sorted Range for Validation ➡ Practice & Master All BST Problems

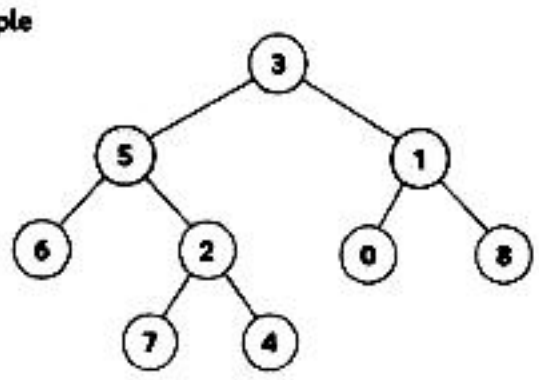
7 ADVANCED TREE PROBLEMS

Important Problems beyond Basics

AHA = Key Insight | Pattern = How to think | Formula / Idea | Complexity

1. LOWEST COMMON ANCESTOR IN BINARY TREE (LC 236)

Example



p = 5, q = 1 → LCA = 3

AHA

Children report back.
If both left and right return non-null → current node is LCA.

Pattern

Postorder (Left → Right → Root)

Idea / Formula

```
If root == null or root == p or root == q → return root
left = recurse(left), right = recurse(right)
If left != null and right != null → return root
Else return left if not null else right
```

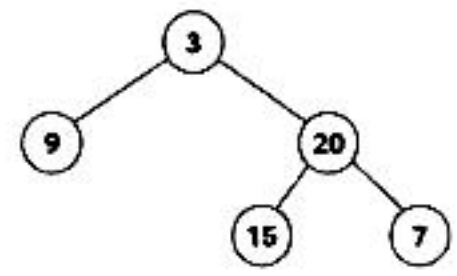
Complexity

Time: O(n) Space: O(h)

2. CONSTRUCT BINARY TREE FROM TRAVERSALS (LC 105)

Example

```
Inorder  :  [9, 3, 15, 20, 7]
Preorder :  [3, 9, 20, 15, 7]
```



AHA

Preorder → Root comes first.
Inorder → Left and Right parts.
Build tree using indices.

Pattern

Recursive (Divide & Conquer)

Idea / Formula

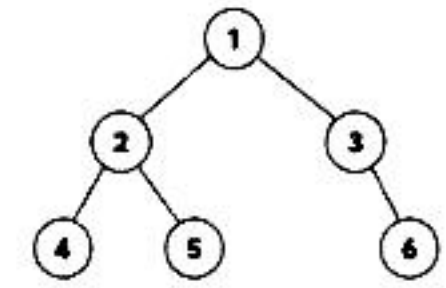
1. First element in preorder = Root
2. Find root in inorder → split left & right
3. Recur for left part and right part

Complexity

Time: O(n) Space: O(n)

3. SERIALIZE AND DESERIALIZE BINARY TREE (LC 297)

Example (Serialize)



Serialized: [1,2,4,null,null,5,3,null,6]
(Level Order with nulls)

AHA

Store structure + values.
Use null marker for missing nodes.

Pattern

BFS (Level Order) or Preorder

Idea / Formula

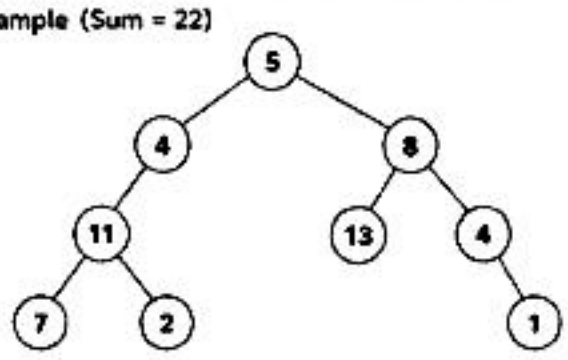
Serialize → Traverse and add values (add nulls)
Deserialize → Rebuild tree from values

Complexity

Time: O(n) Space: O(n)

4. PATH SUM (LC 112 / 113)

Example (Sum = 22)



AHA

Need root to leaf path.
Keep subtracting from target.
If leaf and target == 0 → valid path.

Pattern

DFS (Preorder)

Idea / Formula

target -= node.val
If leaf and target == 0 → true
Else check left and right

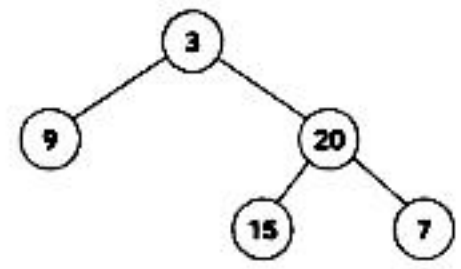
Complexity

Time: O(n) Space: O(h)

5. BUILD TREE FROM INORDER AND POSTORDER (LC 106)

Example

```
Inorder   :  [9, 3, 15, 20, 7]
Postorder :  [9, 15, 7, 20, 3]
```



AHA

Postorder → Root comes last.
Inorder → Left and Right parts.

Pattern

Recursive (Divide & Conquer)

Idea / Formula

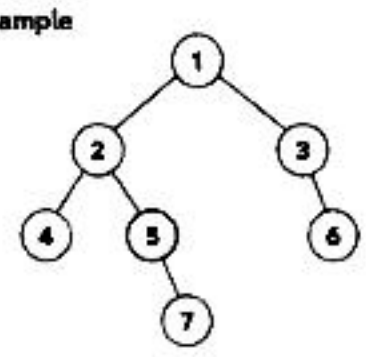
1. Last element in postorder = Root
2. Find root in inorder → split left & right
3. Recur for left part and right part

Complexity

Time: O(n) Space: O(n)

6. BURNING BINARY TREE (LC 2385)

Example



Start Node = 5
Time to burn all nodes = 4

AHA

Fire spreads in all 4 directions.
Convert tree to graph (undirected).
Do BFS from start node.

Pattern

Graph + BFS

Idea / Formula

1. Convert tree to undirected graph
2. BFS from start node
3. Count levels (time)

Complexity

Time: O(n) Space: O(n)

COMMON MENTAL MODELS

Need parent first	Preorder
Need sorted order	Inorder
Need children first	Postorder
Need level by level	BFS
Path / Root to Leaf	DFS (Backtrack)
Multiple directions	Graph + BFS

TREE BUILDING INDEX RULES

Preorder + Inorder

Preorder Root | Left | Right
Inorder Left | Root | Right
→ Use root from preorder

Inorder + Postorder

Inorder Left | Root | Right
Postorder Left | Right | Root
→ Use root from postorder

QUICK RECAP

- Traversal decides the way you think.
- Return type decides the recursion.
- Break into subproblems.
- Combine results.
- Handle base cases carefully.
- Draw the tree. Dry run.
- That's 80% of the battle won!

FINAL MANTRA Understand the Problem ⇒ Choose Pattern ⇒ Draw Tree ⇒ Apply Template ⇒ Code ⇒ Dry Run ⇒ Optimize

8 TREES – FAANG ONE PAGE REVISION

Aha Moments • Patterns • When to use what

1. WHEN TO USE WHICH TRAVERSAL?

Goal / Need	Use This	Why?
Need parent first (decision at node)	Preorder	Process node before children
Need sorted order (BST problems)	Inorder	Left → Node → Right gives sorted order
Need children's information first	Postorder	Process children before node
Need level by level (each level together)	BFS (Level Order)	Visit nodes level by level
Need all paths (root to leaf)	DFS (Backtracking)	Go deep, explore all paths

2. RECURSION – GOLDEN TEMPLATE (DFS)

1. **BASE CASE** — Handle null / edge case
2. **FAITH** — Assume left & right give correct answer
3. **RECUR LEFT** — Recur for left subtree
4. **RECUR RIGHT** — Recur for right subtree
5. **COMBINE** — Use left & right results, do required work
6. **RETURN** — Return the final answer

```
Type solve(Node node) {
  if (node == null)
     return ...;          // 1. Base Case

  Type left  = solve(node.left);  // 3
  Type right = solve(node.right); // 4

  // 5. Combine
  Type ans = ...;

  return ans;                     // 6
}
```

REMEMBER
Return type decides what you can compute.
Break every problem into left & right subproblems.

3. DFS PATTERN RECAP

Problem	Key Idea / Pattern
Max Depth	1 + max(left, right)
Same Tree	left && right
Invert Tree	Swap left & right
Diameter	Through node = left + right
Balanced Tree	|left - right| <= 1

4. BFS PATTERN RECAP

Problem	Key Idea / Pattern
Level Order	Queue + level size
Right Side View	Last node of each level
Zigzag Order	Left to Right / Right to Left toggle
Average of Levels	Sum / size of level
Level Order (N-ary)	Same BFS with N children

5. BST QUICK HITS

- Left Subtree < Node < Right Subtree
- Inorder of BST = Sorted Array
- Search / Insert = Compare & Go
- Delete = 3 cases (0 child, 1 child, 2 children)
- Validate BST = Check range (min, max)
- Kth Smallest = Inorder (kth element)
- LCA in BST = Use BST property

6. COMMON TREE PROBLEM PATTERNS

Pattern	Think
Height / Depth	Add 1 + max(left, right)
Exists / Search	Return true if found in left or right
Count / Sum	Count/Sum from left + right + current
Path Problems	Carry path / sum while going down
Construct Tree	Use Inorder + (Pre/Post)order
Validate / Check	Use constraints / ranges while traversing

7. MENTAL MODELS (AHA MOMENTS)

- Preorder → I do (parent) first
- Inorder → I am in the middle (sorted for BST)
- Postorder → I check results of my children first
- BFS → I explore level by level
- DFS → I go deep and backtrack
- Every node can be a center of diameter
- Tree is a connected acyclic graph
- Remove null checks with base case
- For BST → think in terms of range
- Break problem → Solve small → Combine → Return

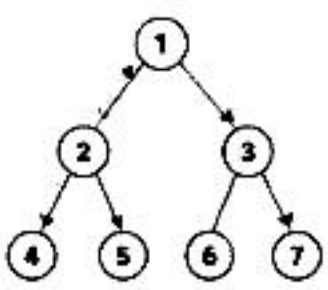
8. COMPLEXITY CHEAT SHEET

Operation / Problem	Time	Space
Traversal (DFS)	O(n)	O(h)
Traversal (BFS)	O(n)	O(w)
Search in BST	O(h)	O(1)
Insert in BST	O(h)	O(1)
Delete in BST	O(h)	O(h)
Height of Tree	O(n)	O(h)
Diameter	O(n)	O(h)

n = number of nodes, h = height, w = max width

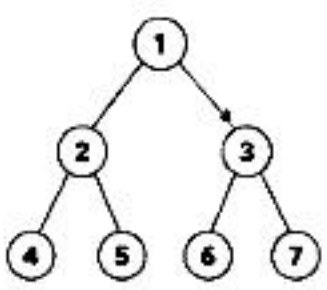
9. TRAVERSAL VISUAL SUMMARY

Preorder (Root → Left → Right)



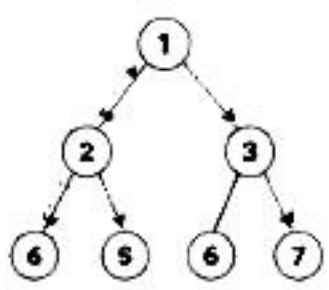
Order: 1, 2, 4, 5, 3, 6, 7

Inorder (Left → Root → Right)



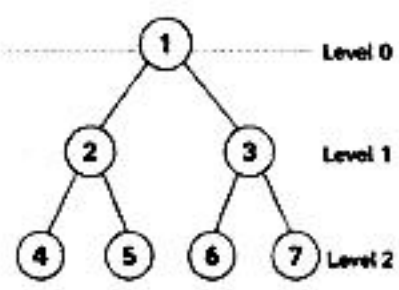
Order: 4, 2, 5, 1, 6, 3, 7

Postorder (Left → Right → Root)



Order: 4, 5, 2, 6, 7, 3, 1

Level Order (BFS) (Level by Level)



Order: 1, 2, 3, 4, 5, 6, 7

10. FINAL MANTRA

- Understand the Problem
- Choose the Right Pattern
- Draw / Dry Run
- Write Clean Code
- Test All Edge Cases
- Optimize if needed

PRACTICE → ANALYZE → REFLECT → IMPROVE → REPEAT

IMPORTANT REMINDERS

Draw the tree Always helps! | Dry run before coding | Handle null carefully | Think recursively, not iteratively | Keep solving, keep improving!